```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv(r"C:\Users\ishus\Downloads\loan_sanction_test.csv")

df
```

```
      Loan_ID Gender Married Dependents     Education Self_Employed  \
0    LP001015   Male     Yes          0      Graduate            No
1    LP001022   Male     Yes          1      Graduate            No
2    LP001031   Male     Yes          2      Graduate            No
3    LP001035   Male     Yes          2      Graduate            No
4    LP001051   Male      No          0  Not Graduate            No
..        ...    ...     ...        ...           ...           ...
362  LP002971   Male     Yes         3+  Not Graduate           Yes
363  LP002975   Male     Yes          0      Graduate            No
364  LP002980   Male      No          0      Graduate            No
365  LP002986   Male     Yes          0      Graduate            No
366  LP002989   Male      No          0      Graduate           Yes

     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term
\
0               5720                  0       110.0             360.0

1               3076               1500       126.0             360.0

2               5000               1800       208.0             360.0

3               2340               2546       100.0             360.0

4               3276                  0        78.0             360.0

..               ...                ...         ...               ...

362             4009               1777       113.0             360.0

363             4158                709       115.0             360.0

364             3250               1993       126.0             360.0

365             5000               2393       158.0             360.0

366             9200                  0        98.0             180.0


     Credit_History Property_Area
0               1.0         Urban
1               1.0         Urban
2               1.0         Urban
3               NaN         Urban
```

```
4                1.0         Urban
..               ...           ...
362              1.0         Urban
363              1.0         Urban
364              NaN     Semiurban
365              1.0         Rural
366              1.0         Rural

[367 rows x 12 columns]
```

Getting few rows of dataset

```
df.head(10)
```

```
    Loan_ID  Gender Married Dependents     Education Self_Employed  \
0  LP001015    Male     Yes          0      Graduate            No
1  LP001022    Male     Yes          1      Graduate            No
2  LP001031    Male     Yes          2      Graduate            No
3  LP001035    Male     Yes          2      Graduate            No
4  LP001051    Male      No          0  Not Graduate            No
5  LP001054    Male     Yes          0  Not Graduate           Yes
6  LP001055  Female      No          1  Not Graduate            No
7  LP001056    Male     Yes          2  Not Graduate            No
8  LP001059    Male     Yes          2      Graduate           NaN
9  LP001067    Male      No          0  Not Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5720                  0       110.0             360.0
1             3076               1500       126.0             360.0
2             5000               1800       208.0             360.0
3             2340               2546       100.0             360.0
4             3276                  0        78.0             360.0
5             2165               3422       152.0             360.0
6             2226                  0        59.0             360.0
7             3881                  0       147.0             360.0
8            13633                  0       280.0             240.0
9             2400               2400       123.0             360.0

   Credit_History Property_Area
0             1.0         Urban
1             1.0         Urban
2             1.0         Urban
3             NaN         Urban
4             1.0         Urban
5             1.0         Urban
6             1.0     Semiurban
7             0.0         Rural
```

```
8              1.0         Urban
9              1.0     Semiurban
```

Getting the info from the dataset which tell us about the numeric and categorical column

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            367 non-null    object
 1   Gender             356 non-null    object
 2   Married            367 non-null    object
 3   Dependents         357 non-null    object
 4   Education          367 non-null    object
 5   Self_Employed      344 non-null    object
 6   ApplicantIncome    367 non-null    int64
 7   CoapplicantIncome  367 non-null    int64
 8   LoanAmount         362 non-null    float64
 9   Loan_Amount_Term   361 non-null    float64
 10  Credit_History     338 non-null    float64
 11  Property_Area      367 non-null    object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

Getting the description of the numerical column

```
#we can say that in Applicant Income and Coapplicant Income column
have high standard deviation which mean they could have outliers
#we all know that if mean is greater than median than the data is
right skewed and vice versa
#after analyzing the numerical column i can say
#ApplicantIncome column is right skewed because mean is greater than
median
#CoapplicantIncome is also right skewed
#LoanAmount is also right skewed but it is mild cause the differnce
between mean and median is small thats why it is mild
#Loan_Amount_Term is left skewed but mild
#

df.describe()
```

```
       ApplicantIncome  CoapplicantIncome  LoanAmount
Loan_Amount_Term  \
count       367.000000         367.000000  362.000000
361.000000
mean       4805.599455        1569.577657  136.132597
342.537396
std        4910.685399        2334.232099   61.366652
65.156643
min           0.000000           0.000000   28.000000
6.000000
25%        2864.000000           0.000000  100.250000
360.000000
50%        3786.000000        1025.000000  125.000000
360.000000
75%        5060.000000        2430.500000  158.000000
360.000000
max       72529.000000       24000.000000  550.000000
480.000000

       Credit_History
count      338.000000
mean         0.825444
std          0.380150
min          0.000000
25%          1.000000
50%          1.000000
75%          1.000000
max          1.000000
```

```
print(df.std(numeric_only = True))
```

```
ApplicantIncome      4910.685399
CoapplicantIncome    2334.232099
LoanAmount             61.366652
Loan_Amount_Term       65.156643
Credit_History          0.380150
dtype: float64
```

```
print(df.median(numeric_only=True))
```

```
ApplicantIncome      3786.0
CoapplicantIncome    1025.0
LoanAmount            125.0
Loan_Amount_Term      360.0
Credit_History          1.0
dtype: float64
```

Filling the nan value in the dataset

```
df.isnull().sum()
```

```
Loan_ID               0
Gender               11
Married               0
Dependents           10
Education             0
Self_Employed        23
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount            5
Loan_Amount_Term      6
Credit_History       29
Property_Area         0
dtype: int64
```

```
df['Gender']=df['Gender'].fillna(method='ffill')
```

```
C:\Users\ishus\AppData\Local\Temp\ipykernel_13496\1412406656.py:1:
FutureWarning: Series.fillna with 'method' is deprecated and will
raise in a future version. Use obj.ffill() or obj.bfill() instead.
  df['Gender']=df['Gender'].fillna(method='ffill')
```

```
df['Dependents'].unique()
```

```
array(['0', '1', '2', '3+', nan], dtype=object)
```

```
df['Dependents']=df['Dependents'].fillna(df['Dependents'].mode()[0])
```

```
df['Self_Employed']=df['Self_Employed'].fillna(df['Self_Employed'].mod
e()[0])
```

```
df['LoanAmount']=df['LoanAmount'].fillna(df['LoanAmount'].mean())
```

```
df['Loan_Amount_Term']=df['Loan_Amount_Term'].fillna(df['Loan_Amount_T
erm'].mean())
```

```
df['Credit_History']=df['Credit_History'].fillna(df['Credit_History'].
mean())
```

```
df.isnull().sum()
```

```
Loan_ID               0
Gender                0
Married               0
Dependents            0
Education             0
Self_Employed         0
ApplicantIncome       0
CoapplicantIncome     0
```

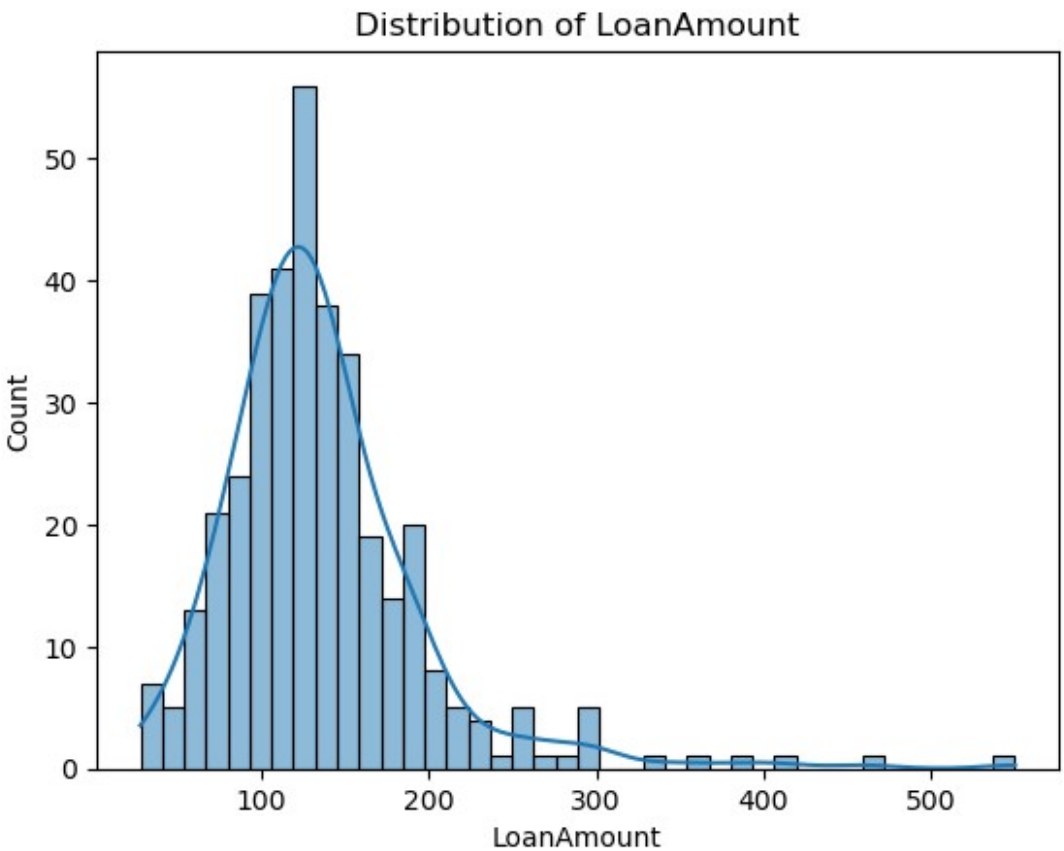
```
LoanAmount            0
Loan_Amount_Term      0
Credit_History        0
Property_Area         0
dtype: int64
```

Univeriate Analysis

for LoanAmount

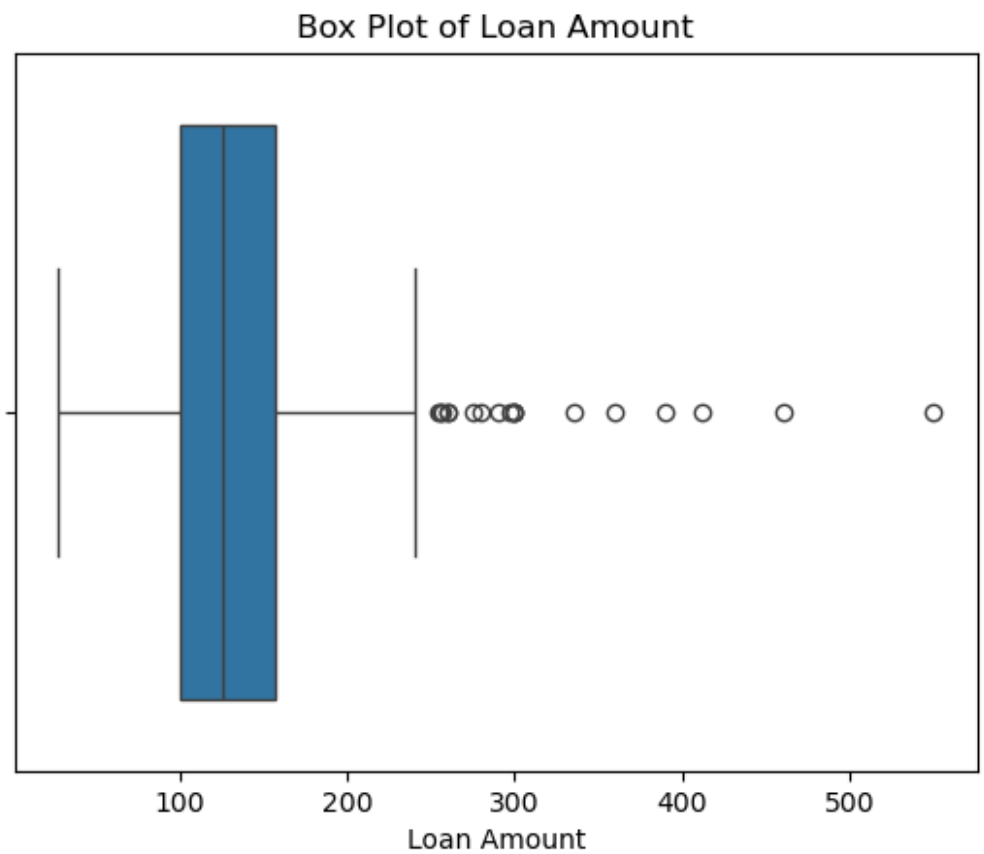
The histogram of LoanAmount shows a right-skewed distribution, where most loan amounts are concentrated in the lower range, with a long tail towards higher values. This indicates the presence of high-value loans that may act as outliers

```
sns.histplot(df['LoanAmount'], bins=40, kde=True)
plt.title('Distribution of LoanAmount')
plt.show()
```



The box plot displays the median, interquartile range, and potential outliers. LoanAmount shows several values beyond the upper whisker, indicating the presence of high-value outliers and a right-skewed distribution.

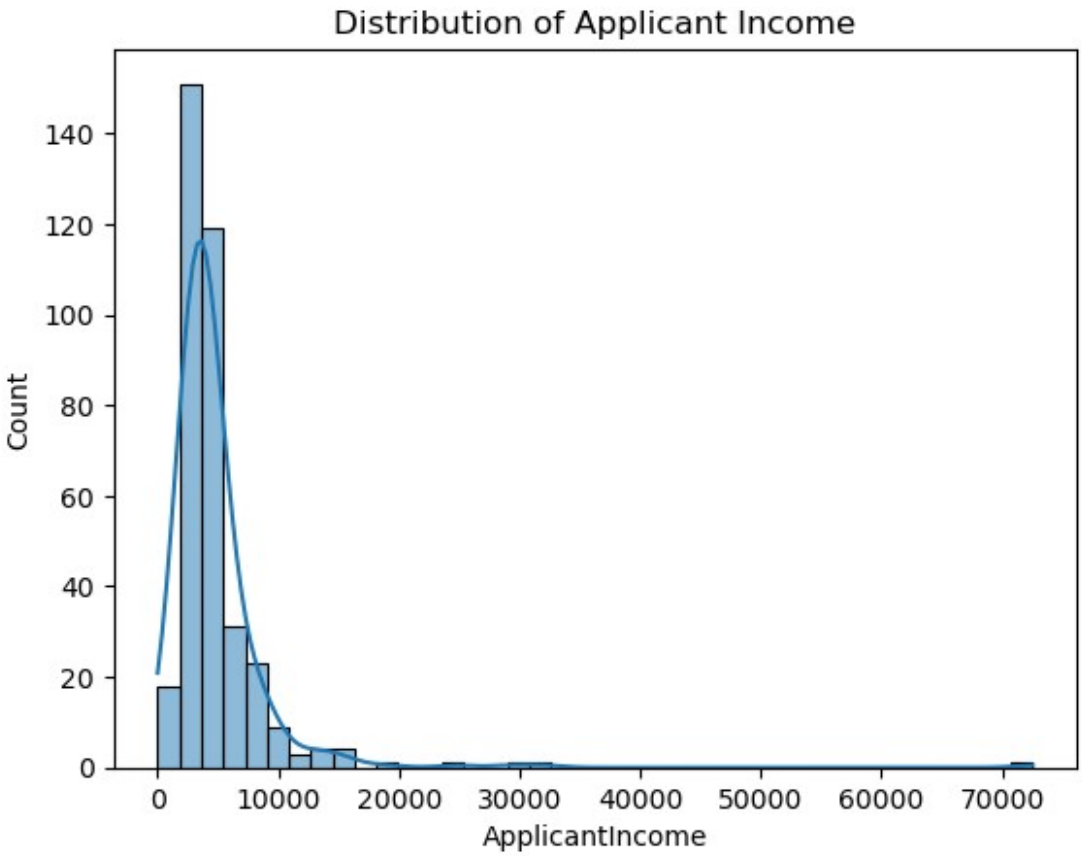
```
sns.boxplot(x=df['LoanAmount'])
plt.title('Box Plot of Loan Amount')
plt.xlabel('Loan Amount')
plt.show()
```



for ApplicantIncome

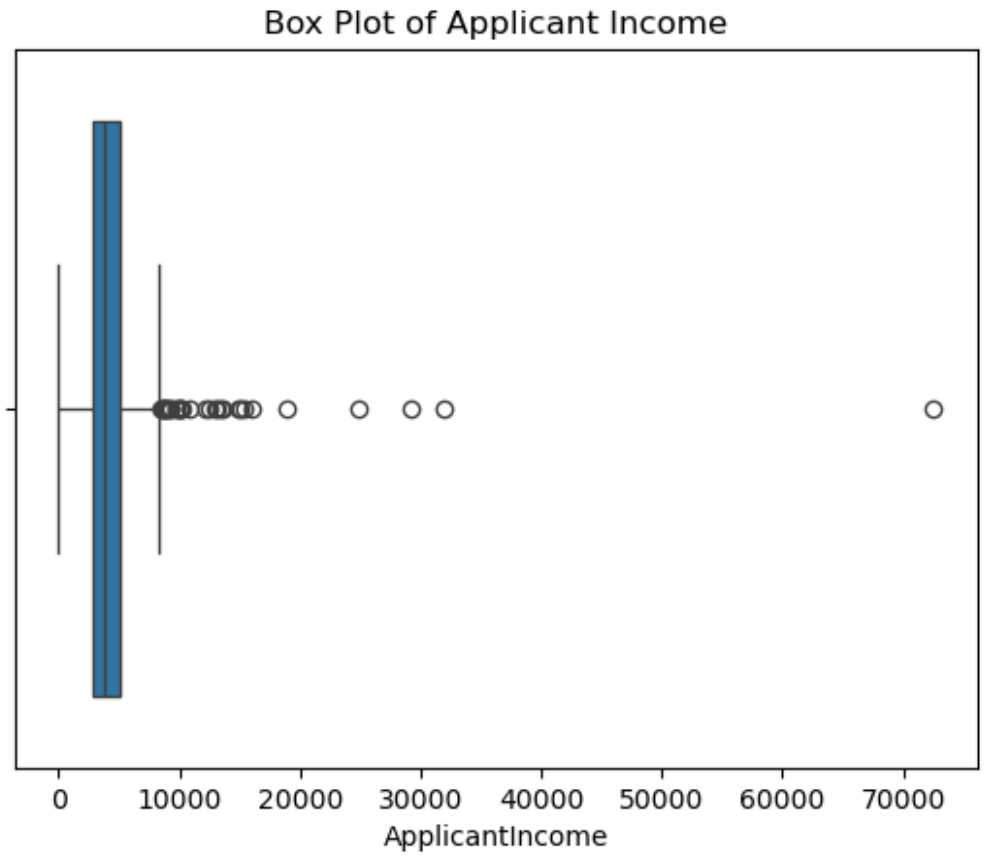
Histogram shows Applicant income is concentrated toward left which mean it is right skewed and long tail toward higher value which refers there exist some outliers

```
sns.histplot(df['ApplicantIncome'], bins=40, kde=True)
plt.title('Distribution of Applicant Income')
plt.show()
```



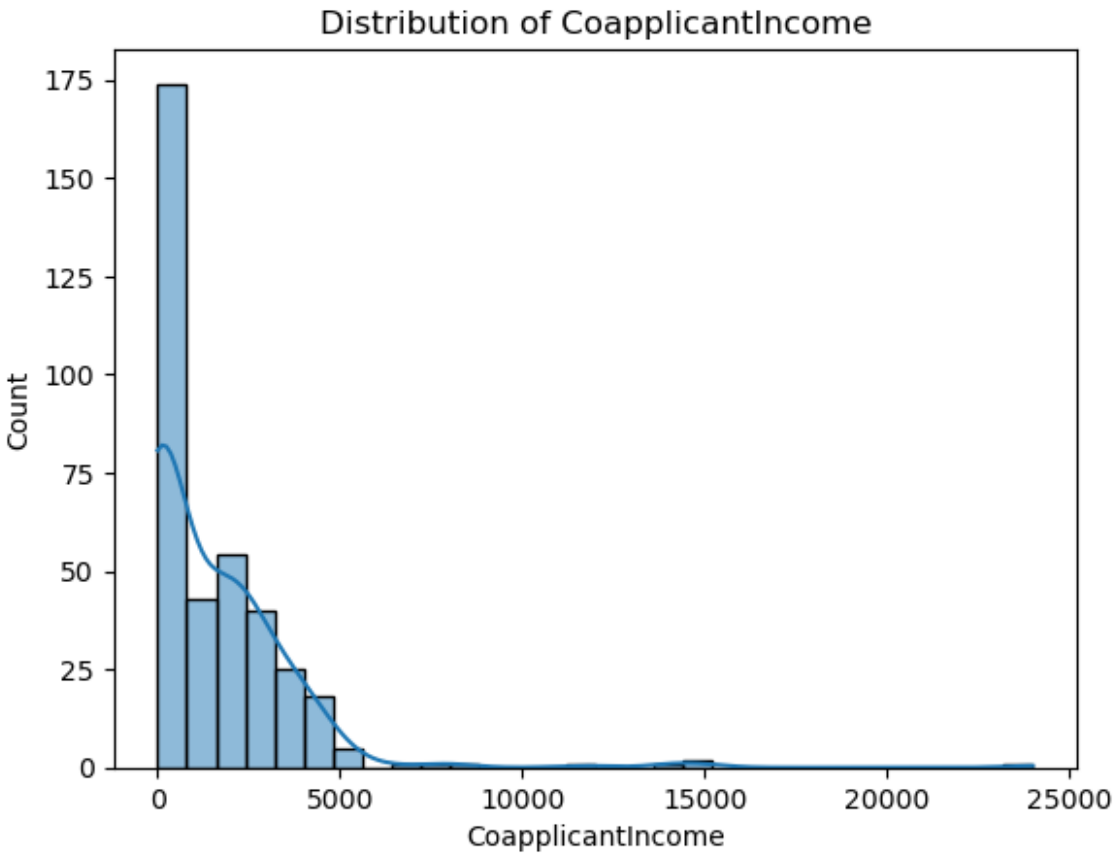
The box plot of ApplicantIncome reveals the presence of several high-value outliers, indicating that a small number of applicants have significantly higher incomes compared to the majority. These outliers are retained in the analysis, as they represent genuine high-income applicants rather than data errors

```
sns.boxplot(x=df['ApplicantIncome'])
plt.title('Box Plot of Applicant Income')
plt.xlabel('ApplicantIncome')
plt.show()
```

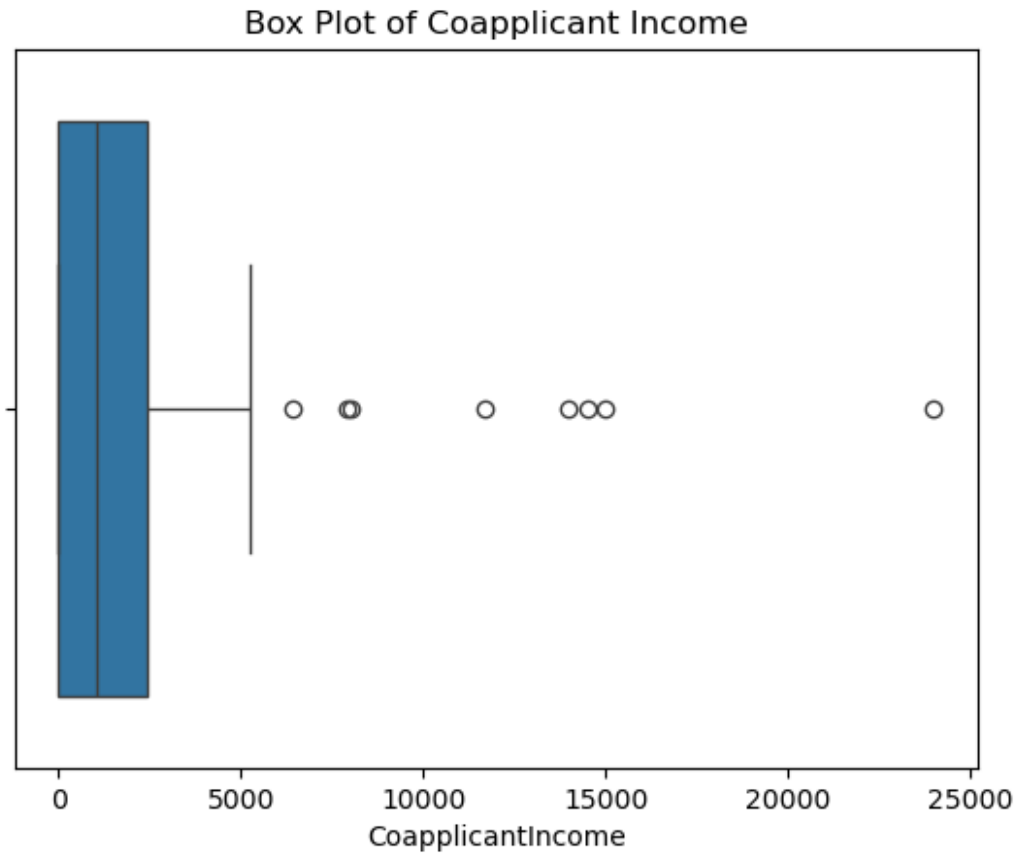


for CoapplicantIncome

```
sns.histplot(df['CoapplicantIncome'], bins=30, kde=True)
plt.title('Distribution of CoapplicantIncome')
plt.show()
```



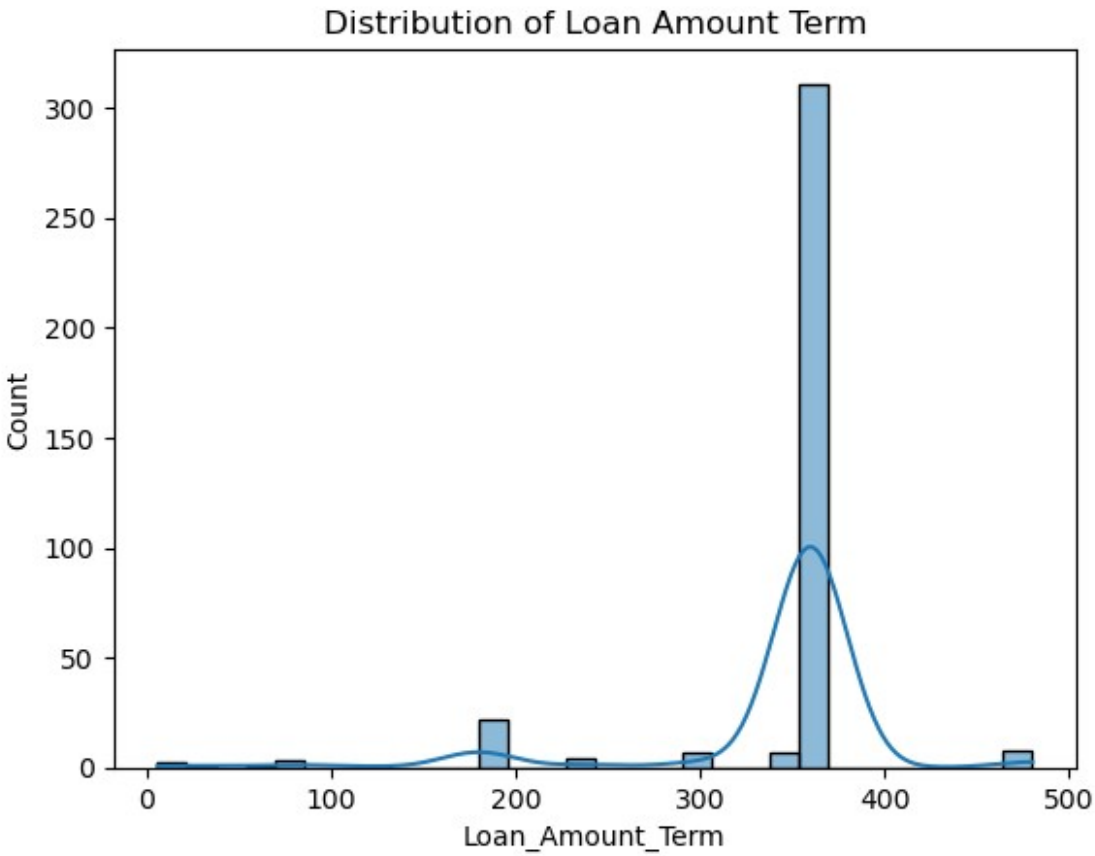

```
sns.boxplot(x=df['CoapplicantIncome'])
plt.title('Box Plot of Coapplicant Income')
plt.xlabel('CoapplicantIncome')
plt.show()
```



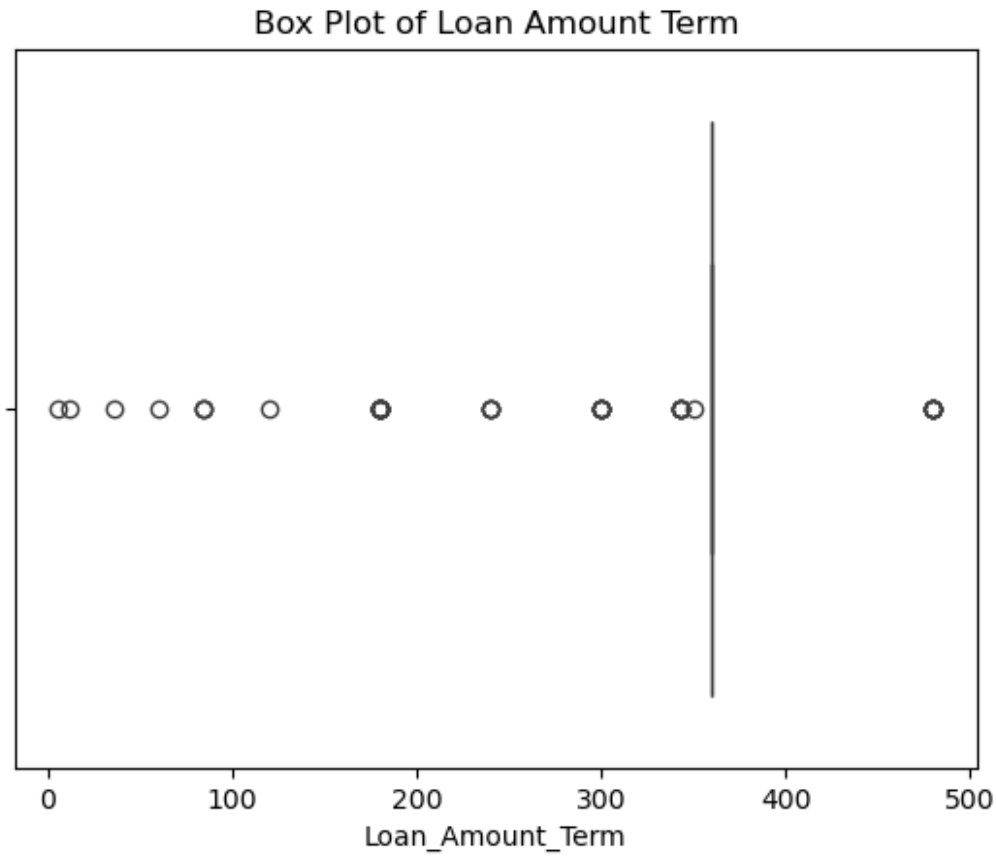
Histogram

We can see it is right skewed from the plot given below and it has some outliers also

```
sns.histplot(df['Loan_Amount_Term'], bins=30, kde=True)
plt.title('Distribution of Loan Amount Term')
plt.show()
```



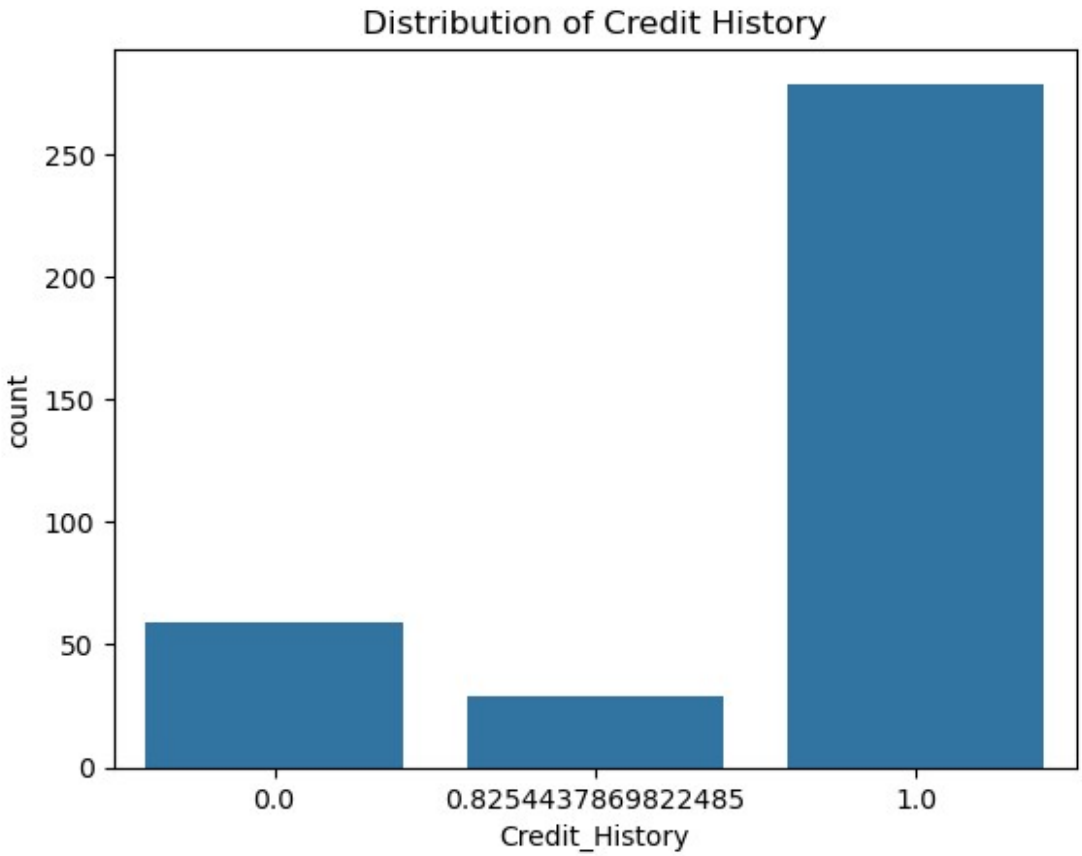

```
sns.boxplot(x=df['Loan_Amount_Term'])
plt.title('Box Plot of Loan Amount Term')
plt.xlabel('Loan_Amount_Term')
plt.show()
```



Credit_History

countplot tells us the variation of credit score between people who are trying to get loan some have great credit history and some are like minimum and some are between both of them

```
sns.countplot(x='Credit_History', data=df)
plt.title('Distribution of Credit History')
plt.show()
```



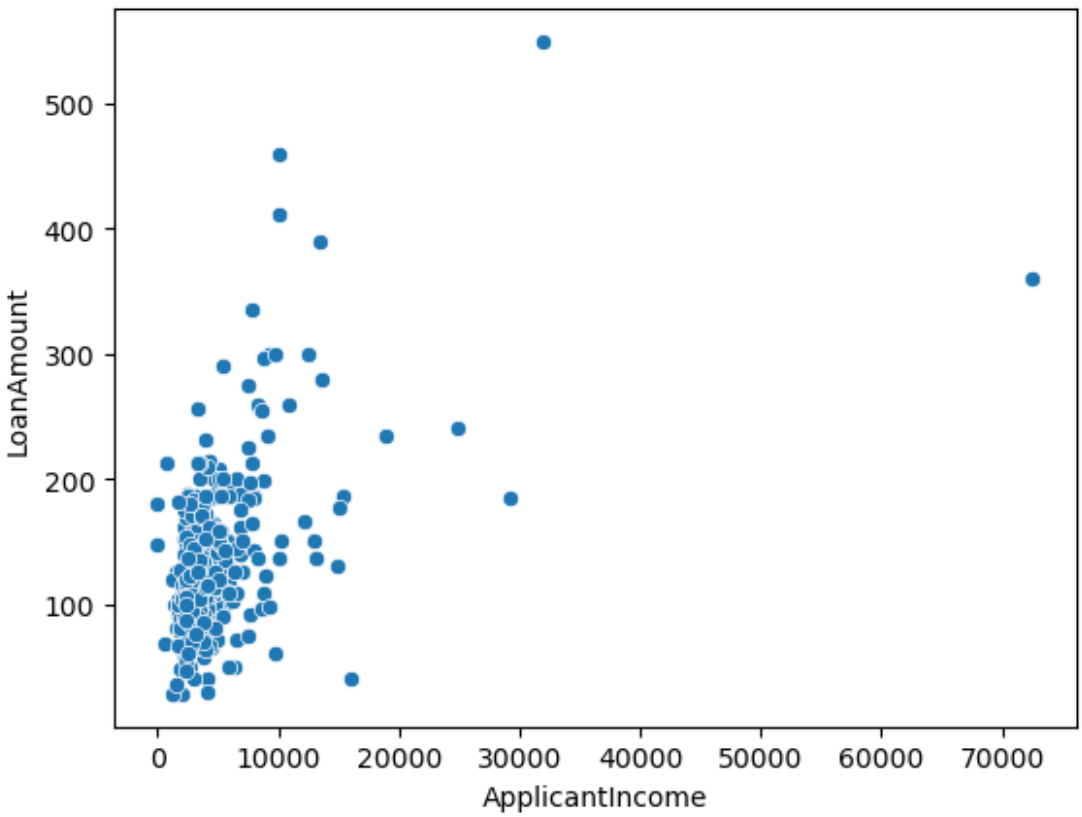
Bivariate Analysis

Applicant Income vs Loan Amount

The scatter plot shows the relationship between ApplicantIncome and LoanAmount. Most applicants fall within the lower income range, and loan amounts are largely concentrated between 100 and 250. The plot indicates a weak positive relationship, suggesting that higher income applicants tend to take slightly higher loan amounts, although the relationship is not very strong due to high variability and presence of outliers.

```
sns.scatterplot(x='ApplicantIncome',y='LoanAmount',data=df)
```

```
<Axes: xlabel='ApplicantIncome', ylabel='LoanAmount'>
```

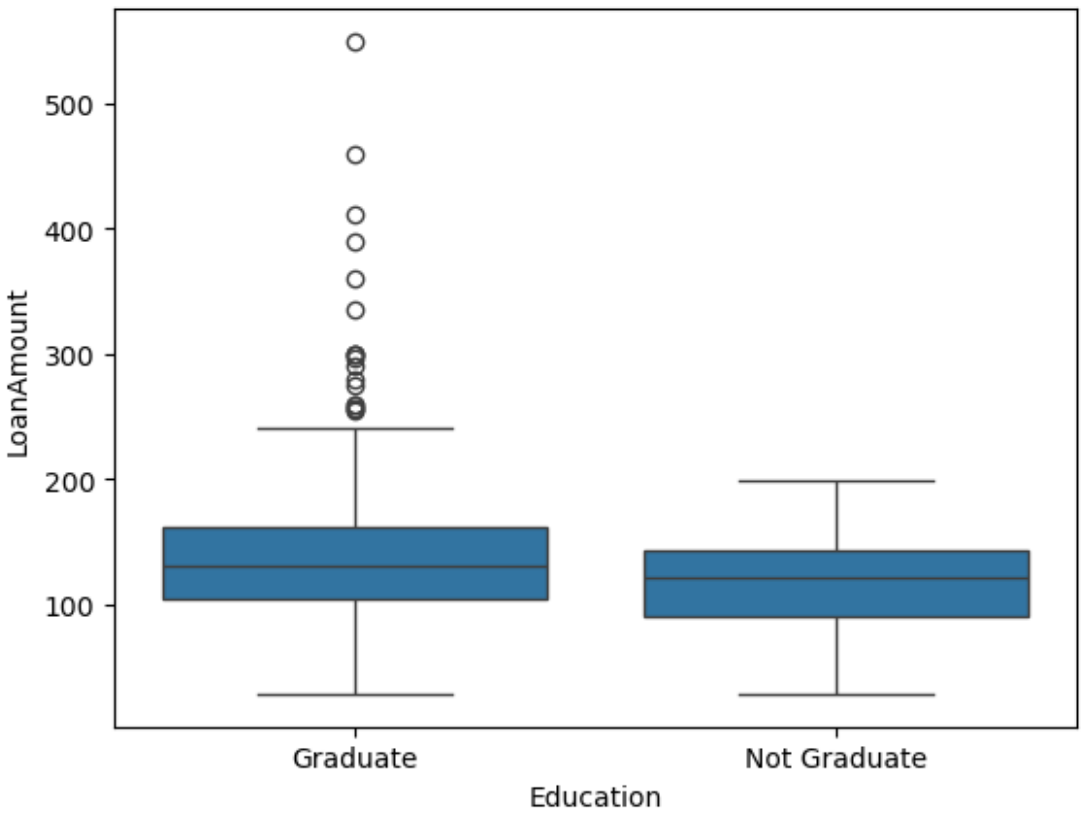


Education vs Loan Amount

The box plot compares LoanAmount across education levels (Graduate and Non-Graduate). Graduate applicants show a slightly higher median loan amount compared to non-graduates; however, the distributions largely overlap, indicating a weak relationship between education and loan amount. A few high-value outliers are observed among graduate applicants, suggesting that some graduates avail larger loans, while non-graduates exhibit a more compact distribution with fewer extreme values.

```
sns.boxplot(x='Education', y='LoanAmount', data=df)
```

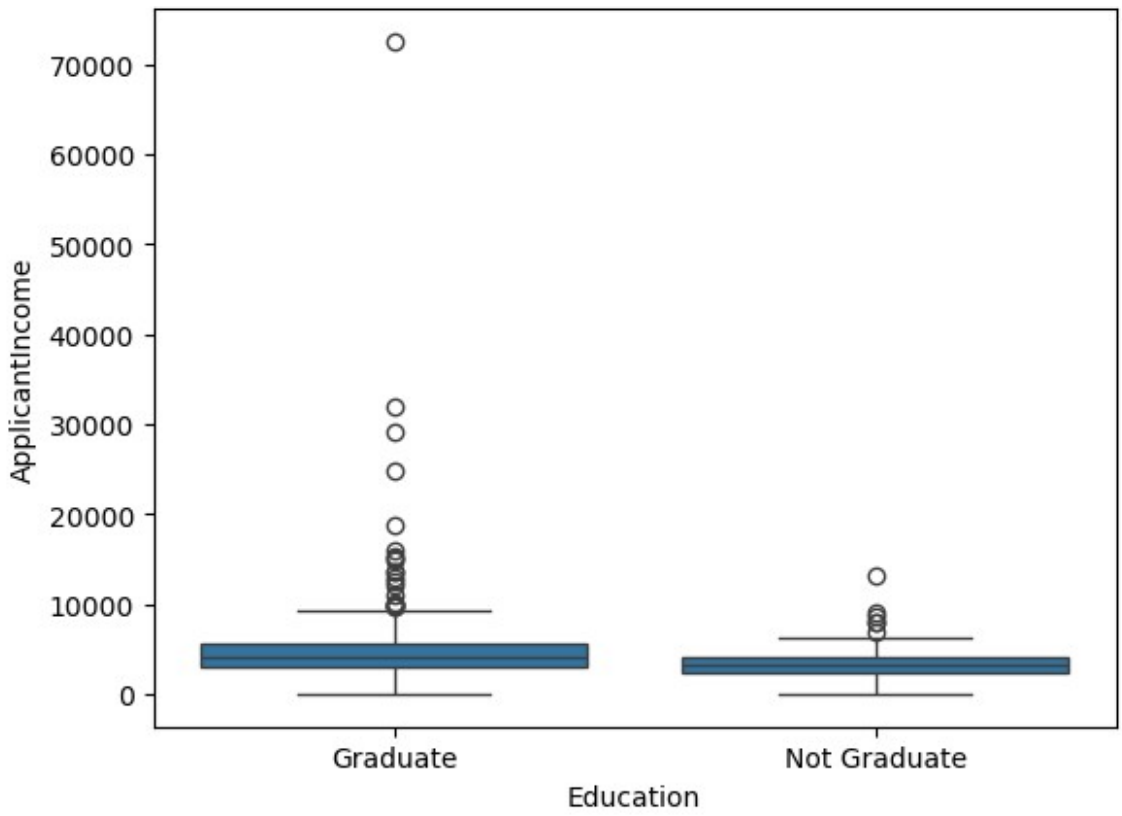
```
<Axes: xlabel='Education', ylabel='LoanAmount'>
```



Education vs ApplicantIncome

The box plot compares ApplicantIncome across education levels (Graduate and Non-Graduate). Graduate applicants show a higher median income compared to non-graduates, along with a wider spread and the presence of high-income outliers. This indicates that education is associated with higher income levels; however, the overlap between the distributions suggests that education alone does not strongly determine applicant income

```
sns.boxplot(x='Education', y='ApplicantIncome', data=df)
plt.show()
```

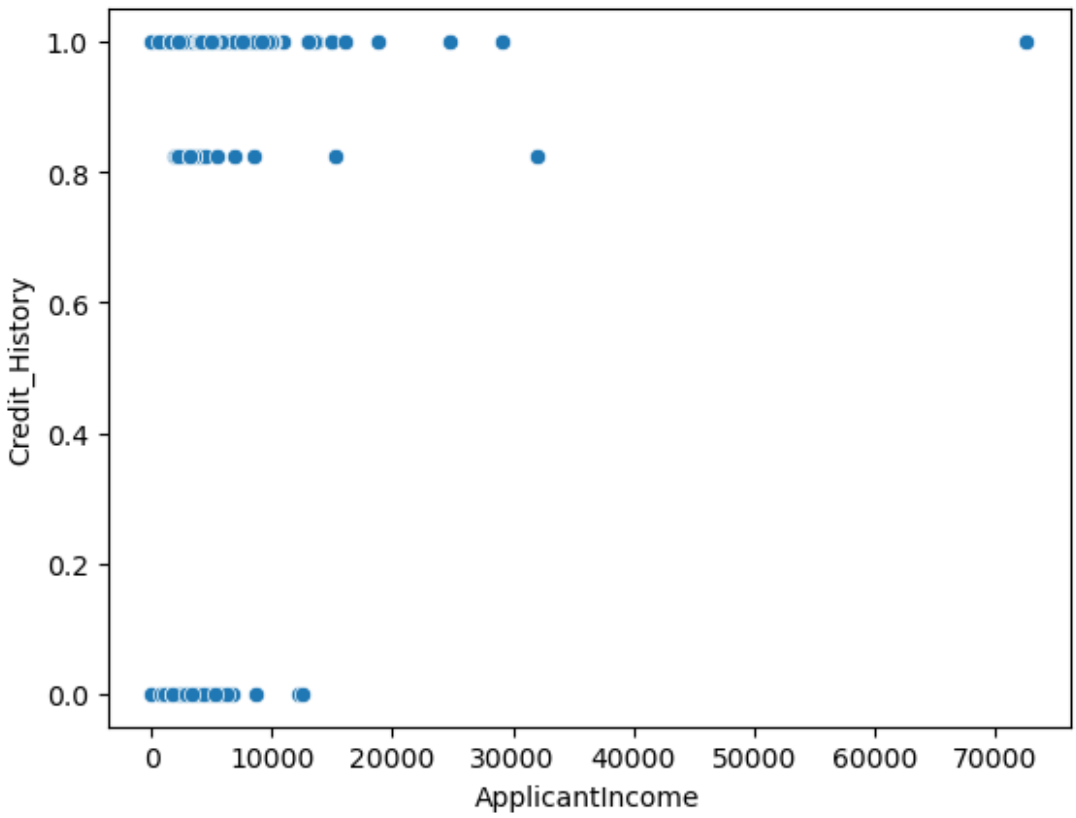


Credit History and Applicant Income

The box plot compares ApplicantIncome across Credit_History categories. Applicants with a positive credit history show a slightly higher median income; however, the overlap between the distributions indicates that income alone does not determine credit history.

```
sns.scatterplot(x='ApplicantIncome',y='Credit_History',data=df)
```

```
<Axes: xlabel='ApplicantIncome', ylabel='Credit_History'>
```



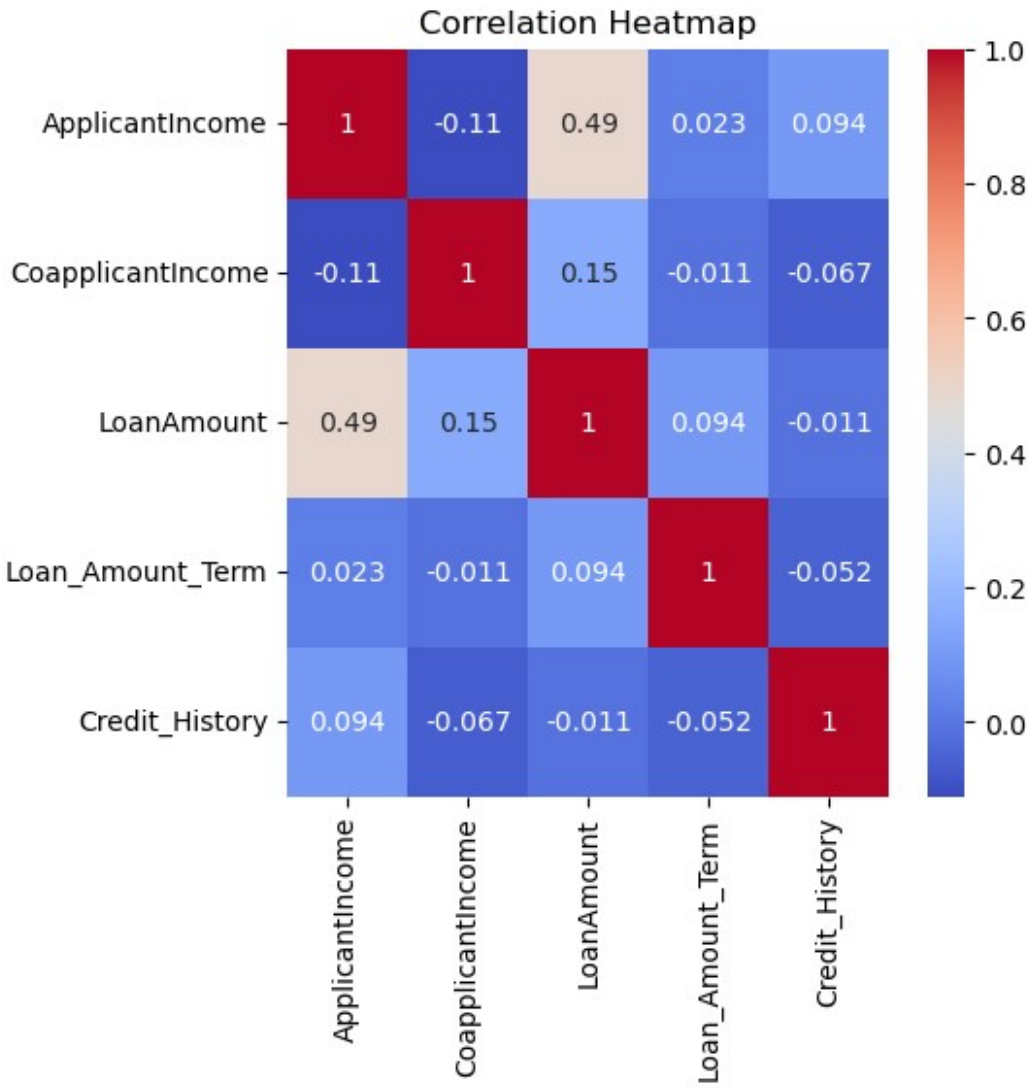
Multivariate Analysis

Multivariate analysis was conducted to examine interactions among multiple variables simultaneously and to uncover deeper patterns that are not evident from univariate or bivariate analysis alone.

Heatmap

The correlation heatmap visualizes the strength and direction of linear relationships among numerical variables, where warmer colors indicate positive correlations and cooler colors indicate negative correlations

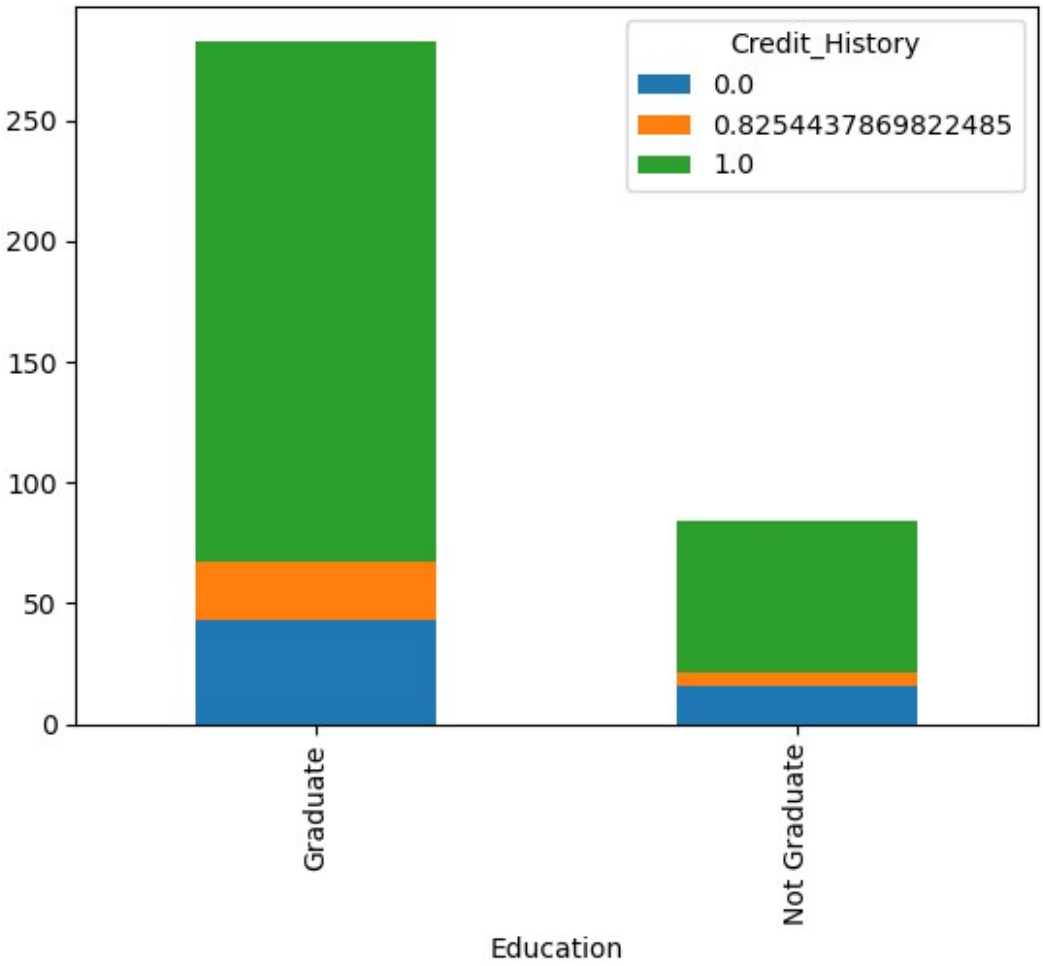
```
corr = df.select_dtypes(include='number').corr()
plt.figure(figsize=(5,5))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



Stacked Bar Chart

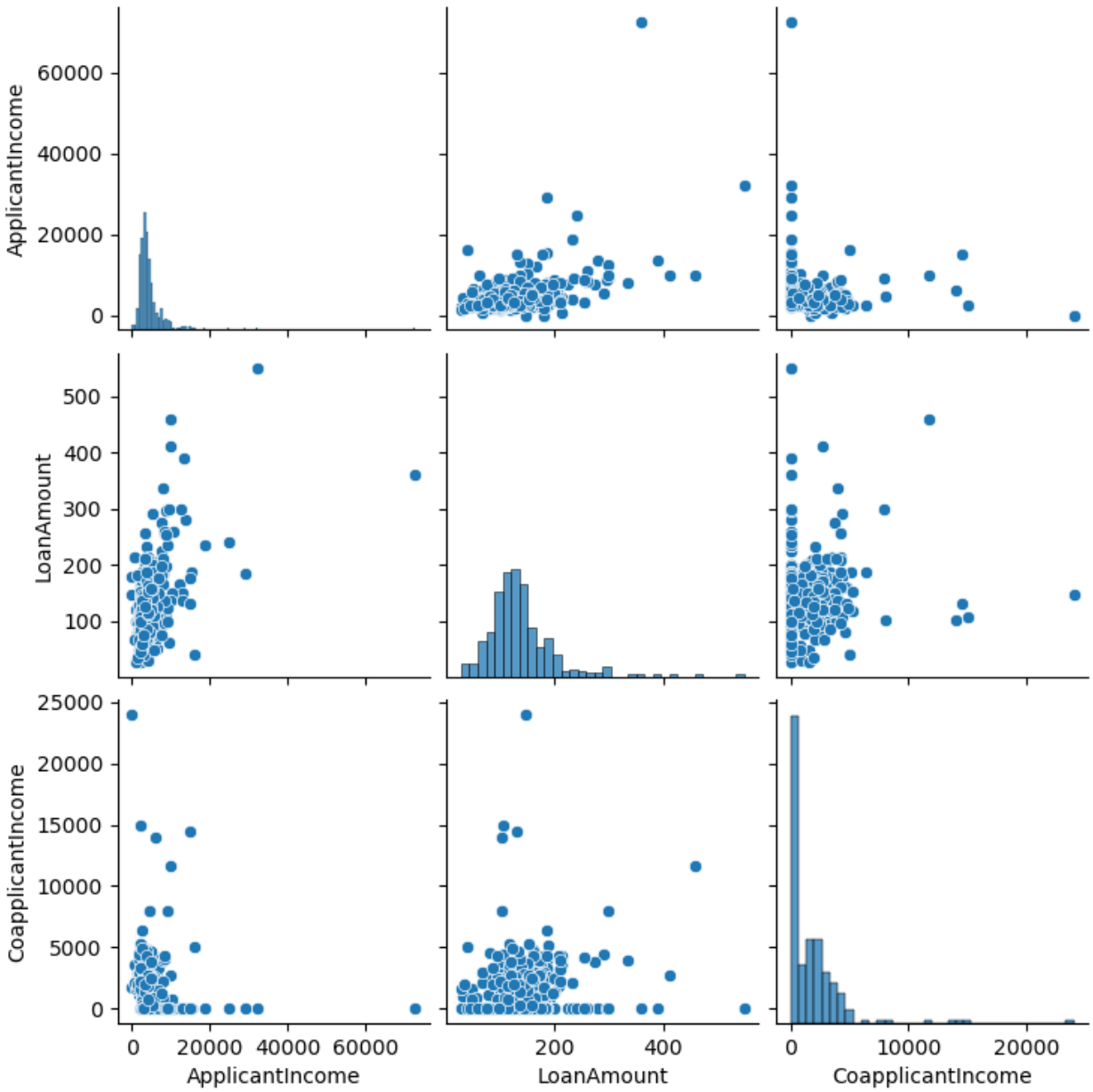
The stacked bar chart shows the distribution of Credit_History across different education levels. Graduate applicants have a higher proportion of positive credit history (value = 1) compared to non-graduates. This suggests that education level may be indirectly associated with credit behavior; however, the presence of both credit history categories within each education group indicates that education alone does not determine credit history

```
pd.crosstab(df['Education'], df['Credit_History']).plot(kind='bar',
stacked=True)
plt.show()
```



Pairplot of ApplicantIncome, LoanAmount, CoapplicantIncome

```
sns.pairplot(df[['ApplicantIncome','LoanAmount','CoapplicantIncome']])
plt.show()
```


ApplicantIncome
LoanAmount
CoapplicantIncome
0
20000
40000
60000
100
200
300
400
500
5000
10000
15000
20000
25000